



NATIONAL SCHOOL OF APPLIED SCIENCES - KHOURLBGA

SULTAN MOULAY SLIMANE UNIVERSITY

PROJECT REPORT

Kairos: Predicting Short-Horizon Price Movements from Limit Order Books with Deep Learning

Presented by :
Achraf ZAHID
Marouane LEMGHARI

Supervised by :
Pr. Fatimaezzahra
CHOUHADI

Academic Year 2025 - 2026

Abstract

We reimplement and adapt the **DeepLOB** architecture (Zhang, Zohren & Roberts, 2019) for live **IEX DEEP** market data, building a complete research pipeline from raw exchange packet captures through to a leakage-controlled trading simulation. The model combines convolutional layers (spatial feature extraction over order-book levels), an Inception module (multi-scale temporal patterns), and an LSTM (temporal dependencies) to classify future price direction at three horizons. We document a central data challenge: because IEX carries only a small fraction of US equity volume, the best-bid/ask mid-price of mega-cap stocks is nearly static, producing degenerate all-flat labels. We solve this with a *deep-book volume-weighted price* as the labelling target and a balanced *quantile* class assignment. The final model reaches a test Macro-F1 of **0.63** (versus a 0.33 random baseline) with predictions distributed across all three classes. A multi-strategy backtest with Kelly sizing and an ensemble of voting risk managers is presented, alongside a frank discussion of why its headline return is dominated by a small number of days and what that implies about the result’s robustness. We also describe the architecture of a production-grade low-latency inference and order-execution system that we designed but were unable to fully implement due to time and resource constraints, representing the primary direction for future development of this work.

Contents

1	Introduction and Intuition	3
1.1	Motivation	3
1.2	Why the Order Book Predicts Price	3
1.3	Why Deep Learning	4
1.4	Project Scope and Structure	4
2	Related Work	5
2.1	Market Microstructure and LOB Predictability	5
2.2	Machine Learning Approaches	5
2.3	The DeepLOB Architecture	5
2.4	Transformer-Based and Attention Approaches	6
2.5	Low-Latency Machine Learning Inference for Trading	6
3	Data and the ETL Pipeline	7
3.1	Source Data	7
3.2	The IEX DEEP Protocol	7
3.3	From Packets to Tensors	8
3.4	Dataset Statistics	9
4	The Central Data Challenge: Flat IEX Prices	10
4.1	Solution: A Deep-Book Volume-Weighted Price	11
5	Labelling: From Prices to Classes	12
5.1	Smoothed-Return Labels	12
5.2	Why We Use Quantiles Rather Than a Fixed Threshold	12
6	Model Architecture	14
6.1	Overview	14
6.2	Component-by-Component Description	14
6.3	Why Three Prediction Horizons	15
7	Training and Evaluation	16
7.1	Protocol and Splits	16
7.2	Why Macro-F1	16
7.3	Training Dynamics	17
7.4	Test Performance	17
8	Backtesting Methodology	19
8.1	Design Philosophy	19

8.2	Signal Fusion and Sizing	19
8.3	Voting Risk Managers	19
8.4	Leakage Control and Test Suite	20
8.5	Configuration	21
9	Backtest Results	22
10	Towards a Production Low-Latency System	24
10.1	Motivation: The Gap Between Research and Deployment	24
10.2	Latency Budget and Requirements	24
10.3	System Architecture	25
10.3.1	Market Data Ingestion	25
10.3.2	Lock-Free LOB Reconstruction	26
10.3.3	Model Inference	26
10.3.4	Order Management and Exchange Connectivity	26
10.3.5	Risk Controls on the Hot Path	26
10.4	Co-location and Infrastructure Considerations	27
10.5	Why This Was Not Completed	27
10.6	Planned Testing Methodology	27
11	Discussion and Honest Limitations	29
11.1	On the Headline Numbers	29
11.2	Principal Limitations	29
11.3	What the Project Does Establish	30
11.4	Summary	30
11.5	Future Work	30

Chapter 1

Introduction and Intuition

1.1 Motivation

Financial markets are among the most information-dense environments humans have constructed. Every moment, thousands of participants submit, modify, and cancel orders that collectively express their beliefs about the fair value of an asset. The **limit order book** (LOB) is the real-time record of all resting buy (bid) and sell (ask) orders for an instrument, organised into discrete price *levels*. It is, in a precise sense, the most granular publicly available description of supply and demand at any instant.

The central hypothesis of this project—well supported in the market-microstructure literature—is that the *shape and evolution* of the order book carries predictive information about imminent price movements. This is not a statement about fundamentals or macroeconomics; it is a statement about the *mechanics* of how price formation actually occurs on an electronic exchange. When large buy orders accumulate at the bid and thin inventory rests on the ask, price is more likely to tick upward. The task, then, is to build a system that reads this signal reliably and quickly.

This report documents our attempt to do so: from raw network packet captures, through a complete engineering pipeline, to a trained deep learning model and a simulated trading environment. We approach every result with deliberate scepticism, pairing each encouraging number with its caveats.

1.2 Why the Order Book Predicts Price

Picture the bid and ask sides of the book as two crowds pressing against a turnstile (the current price). When far more volume rests on the bid than the ask, buyers are “leaning” harder, and the next move is more likely upward. This quantity—**order-book imbalance**—is one of the strongest known short-horizon predictors of price, and has been studied extensively since at least Cont et al. [4] and Gould et al. [3]. The *micro-price* formalises this intuition as a mid-price pulled toward the thinner side of the book:

$$p_{\text{micro}} = \frac{V_{\text{bid}} p_{\text{ask}} + V_{\text{ask}} p_{\text{bid}}}{V_{\text{bid}} + V_{\text{ask}}}. \quad (1.1)$$

Deeper levels matter too. Large resting orders a few ticks away act as walls that price bounces off or breaks through. When a significant offer at level 3 is suddenly cancelled, the order flow dynamics shift in ways that are subtle but learnable. No single hand-crafted feature captures all of these interactions simultaneously; the question is whether a neural network can learn them end-to-end.

1.3 Why Deep Learning

Classical approaches to LOB modelling rely on hand-crafted features: moving averages, volume-at-price histograms, technical indicators derived from level-1 data, or PCA-compressed representations of the book. These embed assumptions, are sensitive to the feature engineer’s domain knowledge, and rarely transfer across instruments or market regimes.

DeepLOB [1] instead learns the mapping directly from raw order-book snapshots. Its convolutional filters are, in effect, *learnable imbalance detectors* across all ten levels and across time, discovered by gradient descent rather than hand-tuned. Crucially, the architecture is specifically designed to respect the two structural axes of the data: the *price-axis* (levels 1 through 10) and the *time-axis* (the sequence of snapshots). Different convolution strides and kernel shapes are applied along each axis accordingly. This is the core motivation for the architecture we adopt and adapt.

1.4 Project Scope and Structure

This report is structured as follows. Chapter 2 surveys related work. Chapter 3 describes the data source and the engineering pipeline. Chapter 4 documents the central methodological challenge introduced by the IEX data source. Chapter 5 explains the labelling scheme. Chapter 6 details the model architecture. Chapter 7 covers training and evaluation. Chapters 8 and 9 present the backtesting design and its results. Chapter 10 describes the planned low-latency production system that we were unable to fully realise within the project timeline. Chapter 11 discusses limitations honestly, and Chapter 12 concludes.

Chapter 2

Related Work

2.1 Market Microstructure and LOB Predictability

The theoretical foundations for LOB-based prediction are well established. Gould et al. [3] provide a comprehensive survey of the limit order book as a stochastic process, characterising its statistical properties across many instruments and exchanges. Cont, Kukanov, and Stoikov [4] demonstrate that the order-book imbalance between the best bid and ask is a statistically significant predictor of the sign of the next mid-price change, even after controlling for trade direction. Their result—that a simple linear combination of level-1 volume imbalance achieves a prediction accuracy significantly above chance in millisecond-resolution data—established the baseline that deep learning approaches now aim to surpass by incorporating deeper levels and longer temporal context.

2.2 Machine Learning Approaches

Early machine learning work applied support vector machines and random forests to hand-crafted LOB features (e.g. best bid/ask spread, depth at each level, volume imbalances). While effective, these methods require careful feature engineering and are sensitive to the choice of features. Kercheval and Zhang [5] provided an influential comparison of SVM and multi-layer perceptron approaches on FI-2010, finding that even shallow neural networks substantially outperform SVMs when given access to the full order book rather than just level-1 data.

Ntakaris et al. [2] introduced the **FI-2010** benchmark dataset, which has since become the standard evaluation platform for LOB prediction models. It consists of 10 days of LOB data from five Finnish stocks on the Helsinki Stock Exchange, with five pre-computed feature representations and three labelling horizons. The availability of this benchmark accelerated the field considerably by enabling direct comparison of approaches.

2.3 The DeepLOB Architecture

Zhang, Zohren, and Roberts [1] introduced DeepLOB, demonstrating that a convolutional-recurrent architecture trained end-to-end on raw LOB data substantially outperforms prior approaches on FI-2010. Their key insights are threefold. First, convolutions with stride-2 along the feature (level) axis naturally learn to combine price and volume at each level, and to aggregate across levels, in a way that mirrors the structure of order-book imbalance. Second, an Inception module [6] with parallel kernel sizes captures temporal patterns at multiple scales simultaneously—analogous to a bank of moving averages with different windows, but learned. Third, an LSTM [7] captures longer-range temporal dependencies across the sequence of snapshots with far fewer parameters than a fully-connected layer of equivalent capacity.

Their reported Macro-F1 on FI-2010 ranges from 0.77 to 0.87 depending on horizon and normalisation scheme. Our adaptation of their architecture to IEX DEEP data, with the modifications described in subsequent chapters, achieves 0.59–0.63 Macro-F1 on a substantially sparser data source—a result we consider non-trivial given the challenge.

2.4 Transformer-Based and Attention Approaches

Since DeepLOB, several groups have explored transformer-based architectures for LOB prediction (e.g. Wallbridge [10]), motivated by the success of self-attention in sequence modelling. These approaches can, in principle, attend directly to specific price levels or time steps that are most informative. However, they typically require more data and longer training to reach competitive performance, which made them less suitable for our constrained dataset. We regard them as a natural future extension of this work.

2.5 Low-Latency Machine Learning Inference for Trading

A largely separate body of literature addresses the engineering challenge of deploying machine learning models with latencies compatible with electronic trading. Agrawal et al. and practitioner sources from exchanges and market-making firms document that meaningful alpha at sub-second horizons requires end-to-end inference latencies measured in microseconds to low milliseconds. Key techniques include kernel-bypass networking (DPDK, RDMA), FPGA-based pre-processing, model quantisation, and batched inference on GPUs co-located at the exchange. We designed an architecture for this regime; its description occupies Chapter 10.

Chapter 3

Data and the ETL Pipeline

3.1 Source Data

We use **IEX DEEP** packet captures (`.pcap`), which record every price-level update on the IEX exchange. IEX (Investors Exchange) is a US equities exchange notable for its “speed bump”—a 350-microsecond intentional delay applied to incoming orders—which was introduced to reduce the advantage of latency-sensitive traders. While this design philosophy makes IEX an interesting subject of study, it also means the exchange carries approximately 2–3% of US equity volume, a fact whose consequences are detailed in Chapter 4.

We process three liquid US equities—**AAPL**, **SPY**, and **NVDA**—over 30 trading days drawn from the July–September 2024 period. This window was chosen deliberately to include the “yen carry-trade unwind” of early August 2024, a period of elevated volatility. Additional out-of-sample sessions extend into Q1 2025. The `.pcap` files are downloaded from IEX’s public data portal and processed entirely locally; no proprietary data is used.

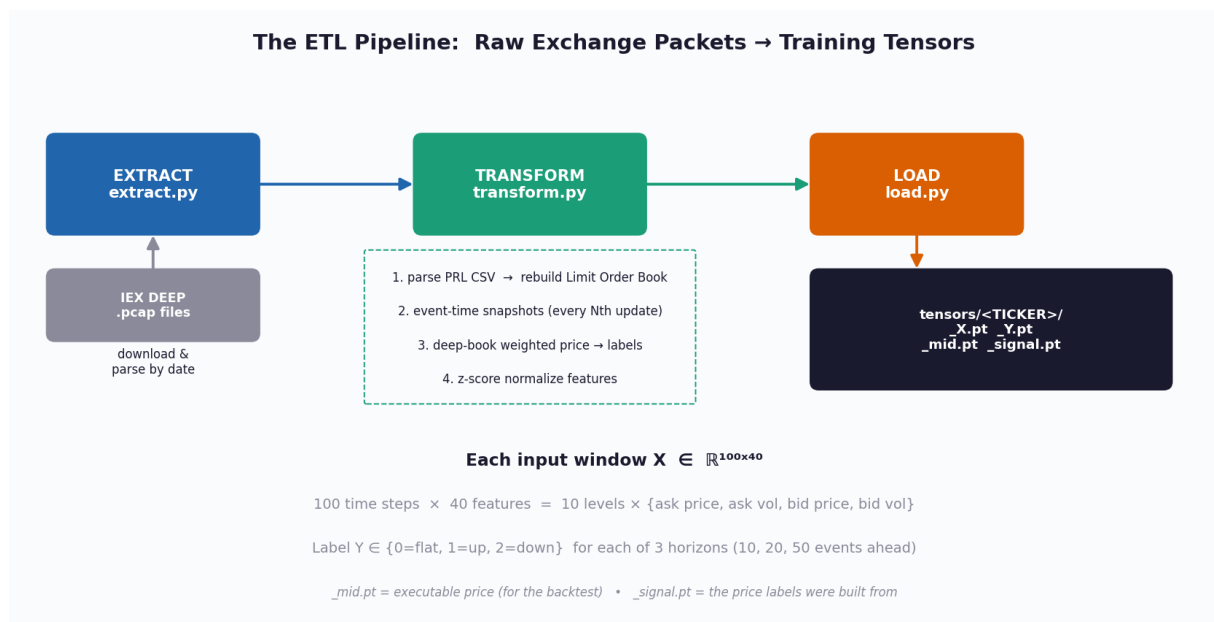


Figure 3.1: The ETL pipeline. Raw IEX packet captures are parsed, replayed to reconstruct the limit order book, snapshotted in event-time, labelled, normalised, and saved as tensors.

3.2 The IEX DEEP Protocol

The IEX DEEP feed transmits a sequence of typed messages over UDP multicast. The relevant message types for LOB reconstruction are:

- **Price Level Update (8145, 0x01 / 0x02)** — adds or modifies a price level; a size of zero signals level removal.
- **Trading Status (0x54)** — indicates auction or halt states that suspend normal order flow.
- **Trade Report (0x54)** — records executed trades, used for validation but not as primary model input.

Messages arrive in sequence-number order. Gaps in sequence numbers indicate dropped packets; we discard days with sequence gaps exceeding a threshold rather than imputing missing levels, as a corrupted book state propagates errors through all downstream snapshots.

3.3 From Packets to Tensors

The **Extract** stage downloads and parses raw `.pcap` files by date and ticker, decoding the binary IEX protocol into a structured stream of update events.

The **Transform** stage performs the core engineering in four steps:

1. **Book reconstruction.** We maintain a sorted dictionary of price levels on each side. Replay every update tick-by-tick, inserting or modifying levels on receipt. Critically, a size of zero is the IEX protocol’s convention for *level removal*—it does not mean “zero quantity resting.” Treating it as a normal update leaves stale prices in the book indefinitely and produces systematically incorrect depth representations. Level-zero messages must trigger a `del` on the dictionary, not an update.
2. **Event-time snapshots.** Rather than snapshotting once per wall-clock second—which on a sparse feed like IEX yields thousands of identical consecutive rows carrying no new information—we emit a snapshot every N book-update events. Every row in the tensor therefore carries genuinely new state. The parameter N is a hyperparameter; we use $N = 10$ as a balance between temporal resolution and tensor size.
3. **Labelling** (described fully in Chapter 5).
4. **Normalisation.** Each of the 40 features is z -scored independently across the day’s snapshots ($\mu = 0, \sigma = 1$). This ensures that prices and volumes, which differ by orders of magnitude, contribute comparably to the convolution responses, and prevents the model from memorising absolute price levels that change from day to day.

The **Load** stage writes four tensors per ticker-day to disk. Each model input is a window $X \in R^{100 \times 40}$:

$$X = [x_1, \dots, x_{100}]^\top, \quad x_t = \{p_a^{(i)}, v_a^{(i)}, p_b^{(i)}, v_b^{(i)}\}_{i=1}^{10}, \quad (3.1)$$

i.e. the 100 most recent order-book states, each with $10 \text{ levels} \times (\text{ask price, ask volume, bid price, bid volume}) = 40$ features.

Table 3.1: Tensors written per ticker-day.

File	Shape	Purpose
<code>_X.pt</code>	$(N, 100, 40)$	Model inputs (normalised)
<code>_Y.pt</code>	$(N, 3)$	Labels for horizons 10/20/50
<code>_mid.pt</code>	$(T,)$	Executable mid-price (used by the back-tester)
<code>_signal.pt</code>	$(T,)$	Price series the labels were derived from

3.4 Dataset Statistics

After filtering days with sequence gaps or insufficient activity (fewer than 500 snapshots), the final dataset spans 28 usable days for training/validation and 10 for testing. Table 3.2 summarises the per-ticker counts.

Table 3.2: Dataset summary after quality filtering.

Ticker	Train+Val days	Test days	Avg. snapshots/day
AAPL	28	10	$\approx 4,200$
SPY	28	10	$\approx 5,800$
NVDA	28	10	$\approx 3,900$

Chapter 4

The Central Data Challenge: Flat IEX Prices

This chapter explains the most important methodological decision in the project.

The original DeepLOB paper used London Stock Exchange data—roughly 150,000 events per stock per day, with the mid-price in near-constant motion. **IEX, by contrast, carries only about 2–3% of US equity volume.** For mega-caps such as AAPL and NVDA, the best bid/ask therefore *barely changes all day*; we measured as few as 2–3 mid-price changes across an entire session. The reason is straightforward: because AAPL’s national best bid and offer (NBBO) is determined by the aggregate of all US equity exchanges, and IEX contributes only a small fraction of that volume, the price that appears in IEX’s own level-1 is rarely the NBBO—and when it is, it reflects a consensus across the whole market, not IEX-specific activity. The IEX book’s true information lives in the *depth*, not the top of book.

A label built on a price that never moves is always “flat.” Trained on $\sim 99.9\%$ flat labels, a model learns the only rational policy—always predict flat—achieving high accuracy while being completely useless. We confirmed this failure mode empirically: early configurations collapsed to a single predicted class, scoring 0.33–0.34 Macro-F1 (consistent with a pure-flat predictor on balanced classes) regardless of training duration.

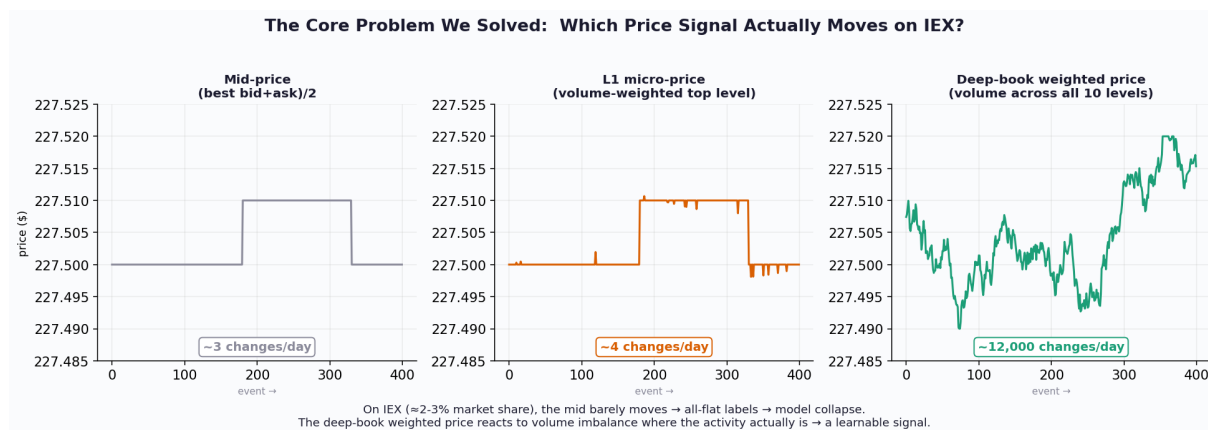


Figure 4.1: Three candidate label signals on representative IEX data. The mid-price and the level-1 micro-price are nearly frozen; the deep-book volume-weighted price reacts to the deep-level activity where IEX’s information actually lives.

4.1 Solution: A Deep-Book Volume-Weighted Price

We weight the best bid/ask prices by the *total* volume across all ten visible levels:

$$p_{\text{signal}} = \frac{V_{\text{bid}}^{(1-10)} p_{\text{ask}}^{(1)} + V_{\text{ask}}^{(1-10)} p_{\text{bid}}^{(1)}}{V_{\text{bid}}^{(1-10)} + V_{\text{ask}}^{(1-10)}}. \quad (4.1)$$

This extends the imbalance concept from one level to ten, capturing the deep-level churn that constitutes the bulk of IEX activity. Empirically it changes thousands of times per day where the mid changes only a handful, yielding a genuinely learnable target. It remains faithful to the paper’s thesis that imbalance predicts price—we simply measure imbalance across the whole visible book rather than just level 1.

The intuition is that when, say, 80% of the total bid-side depth suddenly concentrates at levels 7–10 (far below the best bid), the book has become top-heavy on the ask side even if the level-1 spread has not moved. This pressure imbalance is a real microstructure signal that the standard mid-price does not encode.

Note

Interpretation note. The model therefore predicts the direction of the *book-pressure-weighted price*, not the raw national-best-bid-offer mid. On IEX this is the only target with sufficient movement to learn from. With full-market data (e.g. NASDAQ ITCH) one would revert to the mid, and our ETL pipeline supports this switch via a single configuration flag.

Chapter 5

Labelling: From Prices to Classes

5.1 Smoothed-Return Labels

We never compare a single instant to a single future instant—individual price changes at this granularity are dominated by microstructure noise (bid-ask bounce, order rounding). Following the paper (Eq. 4), we compare *smoothed* windows of past and future prices:

$$m_-(t) = \frac{1}{k} \sum_{i=0}^k p_{t-i}, \quad m_+(t) = \frac{1}{k} \sum_{i=1}^k p_{t+i}, \quad \ell_t = \frac{m_+(t) - m_-(t)}{m_-(t)}. \quad (5.1)$$

The smoothing window k matches the prediction horizon $h \in \{10, 20, 50\}$. The quantity ℓ_t is a normalised, noise-reduced measure of price trend over a window of h future events.

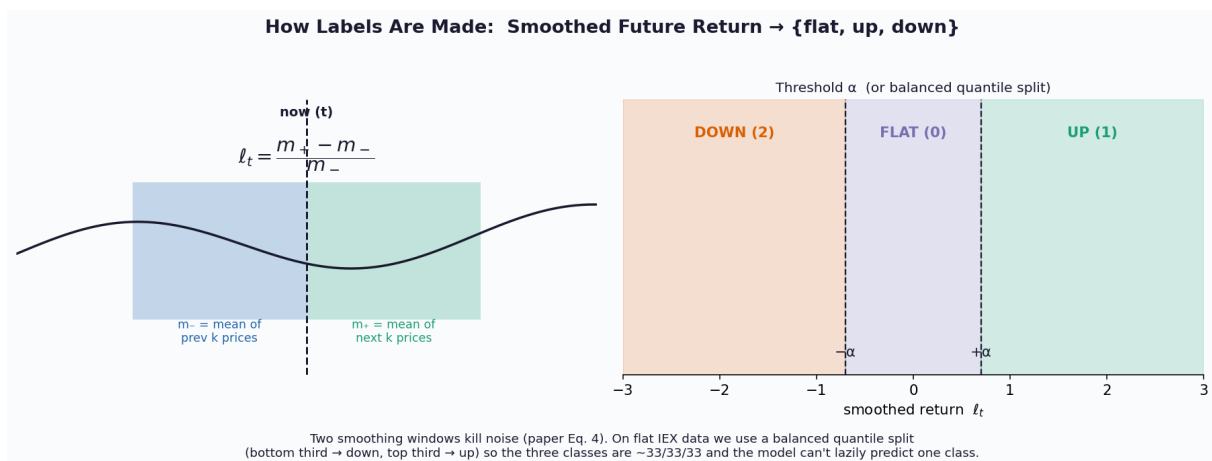


Figure 5.1: Labelling. Left: past (m_-) and future (m_+) smoothing windows around time t yield a smoothed return ℓ_t . Right: ℓ_t is mapped to a class by a fixed threshold α or, on sparse data, a balanced quantile split.

5.2 Why We Use Quantiles Rather Than a Fixed Threshold

A fixed threshold α classifies $\ell_t > \alpha$ as UP, $\ell_t < -\alpha$ as DOWN, else FLAT. On sparse data this *also* collapses to all-flat, because the overwhelming mass of ℓ_t values is essentially zero: if the signal moves by only one tick per day, 99.9% of smoothed returns are numerically indistinguishable.

We therefore default to a **balanced quantile split**: rank all returns per horizon and label the bottom 33rd percentile DOWN, the top 33rd percentile UP, and the middle third FLAT. This guarantees an approximately 33/33/33 class balance *by construction*, which

forces the model to discriminate rather than exploit a majority class, and makes Macro-F1 a meaningful, interpretable metric (random = 0.333).

Note

The quantile scheme forces balance across the *training distribution*. A consequence is that the class boundaries are data-dependent: the threshold for what counts as “up” varies from day to day and ticker to ticker, meaning the labels are calibrated to relative movement within each session, not to any absolute price threshold. This is a deliberate choice—it ensures trainability—but it means the model’s predictions are relative, not absolute. The fixed-threshold method remains available for denser data sources.

Chapter 6

Model Architecture

6.1 Overview

The DeepLOB architecture comprises four sequential components, each responsible for a distinct stage of representation learning. Figure 6.1 illustrates the full network, which has approximately 79,000 trainable parameters.

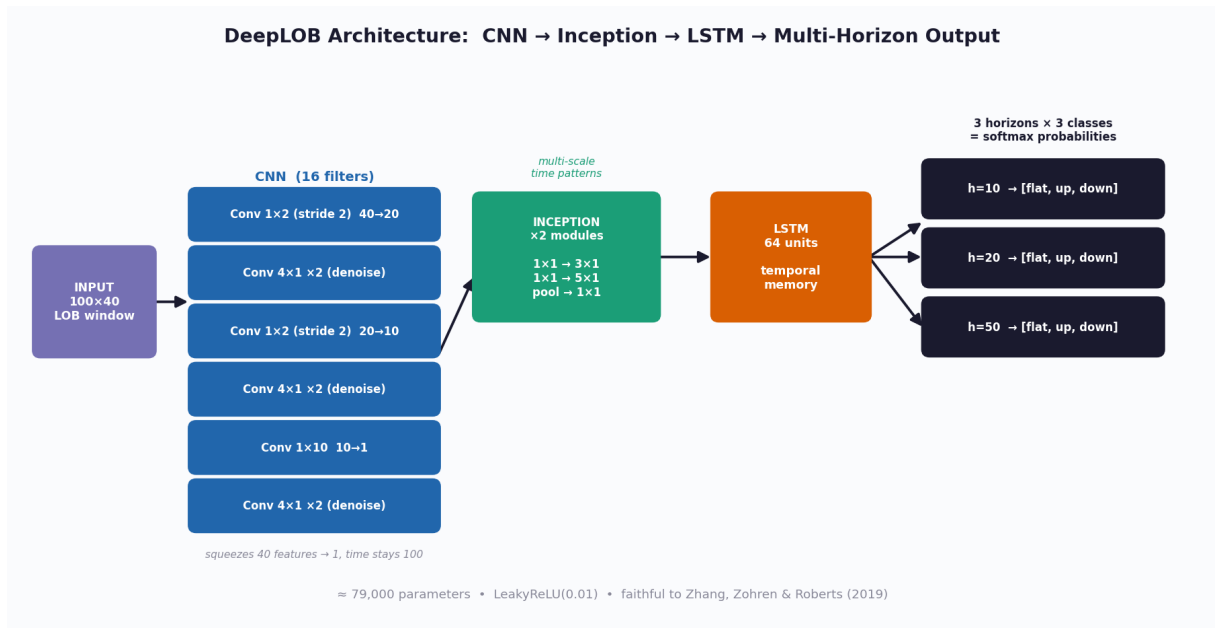


Figure 6.1: The DeepLOB network ($\approx 79k$ parameters): a CNN stack squeezes the 40 features to one while preserving the time axis, two Inception modules capture multi-scale temporal structure, and an LSTM models temporal dependencies before a multi-horizon classification head.

6.2 Component-by-Component Description

CNN Stack (16 filters, 9 convolutional layers). The first 1×2 stride-2 convolution combines {price, volume} at each level into a single latent feature per level—effectively learning a weighted combination of price and volume imbalance at each depth. A second strided 1×2 convolution combines adjacent levels, gradually integrating information across the depth dimension. A final 1×10 convolution collapses all remaining levels to a single vector per time step. Interleaved 4×1 convolutions act as FIR (finite impulse response) denoising filters over time, smoothing each latent channel independently. Crucially, zero-padding preserves the time dimension at 100 throughout this stack: *where* in the 100-step window a pattern occurs is itself informative (patterns near the

end of the window are more recent), so no temporal pooling is applied.

Inception Module ($\times 2$). Two stacked Inception blocks each apply three parallel paths to the time axis: a 3×1 convolution, a 5×1 convolution, and a max-pooling path, all preceded by 1×1 bottleneck convolutions to control channel width. The outputs are concatenated along the channel axis. The effect is that the block simultaneously detects patterns at short ($k = 3$), medium ($k = 5$), and aggregated-short timescales, analogous to a bank of moving averages with different window lengths—but with the window lengths and weights learned by gradient descent rather than fixed by the practitioner.

LSTM (64 units). The output of the Inception stack is a sequence of 100 feature vectors, one per time step. The LSTM [7] processes this sequence, maintaining a hidden state that can span the full 100-step history. It replaces the large fully-connected layer that would otherwise be required, achieving similar representational capacity with roughly $10\times$ fewer parameters and substantially lower overfitting risk on short financial time series. Only the final hidden state is used downstream, serving as a compressed summary of the entire 100-step window.

Multi-Horizon Classification Head. A single linear layer maps the 64-dimensional LSTM output to $3 \text{ horizons} \times 3 \text{ classes} = 9$ logits, followed by per-horizon softmax activations. This yields calibrated (in the entropy sense, though not probabilistically calibrated) distributions over $\{\text{DOWN}, \text{FLAT}, \text{UP}\}$ at horizons $h = 10, 20$, and 50 .

6.3 Why Three Prediction Horizons

Training across multiple horizons simultaneously serves two purposes. First, it is a form of **multi-task regularisation**: the shared CNN and LSTM representations must be useful for prediction at all three timescales, which discourages the network from overfitting to idiosyncrasies of any single horizon. Second, it provides richer trading signal: different horizons complement each other in the backtester’s signal fusion (Section 8.2), with shorter horizons providing faster reaction and longer horizons providing more robust confirmation.

Chapter 7

Training and Evaluation

7.1 Protocol and Splits

We use strict **chronological splits** to prevent any form of look-ahead. The earliest 20 days per ticker form the training set, the next 5 form the validation set, and the final 10 are the held-out test set. No shuffling is applied across the day boundary; within a day, the sequence of snapshots is preserved in order. This is the only correct procedure for financial time series, where any cross-sectional or random split would introduce future information into the training distribution (a form of data leakage).

The loss function is a class-weighted, multi-task cross-entropy, summed over all three horizons:

$$\mathcal{L} = \sum_{h \in \{10, 20, 50\}} \sum_t w_{y_t} \log P_h(y_t | X_t), \quad (7.1)$$

where w_{y_t} is the inverse-frequency weight of class y_t within the training batch. Even though quantile labels are approximately balanced, small deviations in each batch are corrected by this weighting to prevent any class from dominating gradients.

Optimisation uses Adam with a learning rate of 1×10^{-3} , decayed by a factor of 0.5 if validation Macro-F1 does not improve for 5 consecutive epochs. Gradient clipping at $\text{norm} = 1.0$ prevents the exploding-gradient pathology to which LSTMs are susceptible on high-variance financial data. Models are checkpointed whenever validation Macro-F1 improves above the collapse floor of 0.34.

7.2 Why Macro-F1

On balanced three-class data, **Macro-F1** (the unweighted mean of per-class F1) is the honest scoreboard. A model that lazily predicts one class scores ≈ 0.33 , identical to random. Accuracy alone rewards a degenerate majority predictor; Macro-F1 does not. Precision and recall are reported separately to distinguish between models that are conservative-but-precise and those that are aggressive-but-noisy.

7.3 Training Dynamics

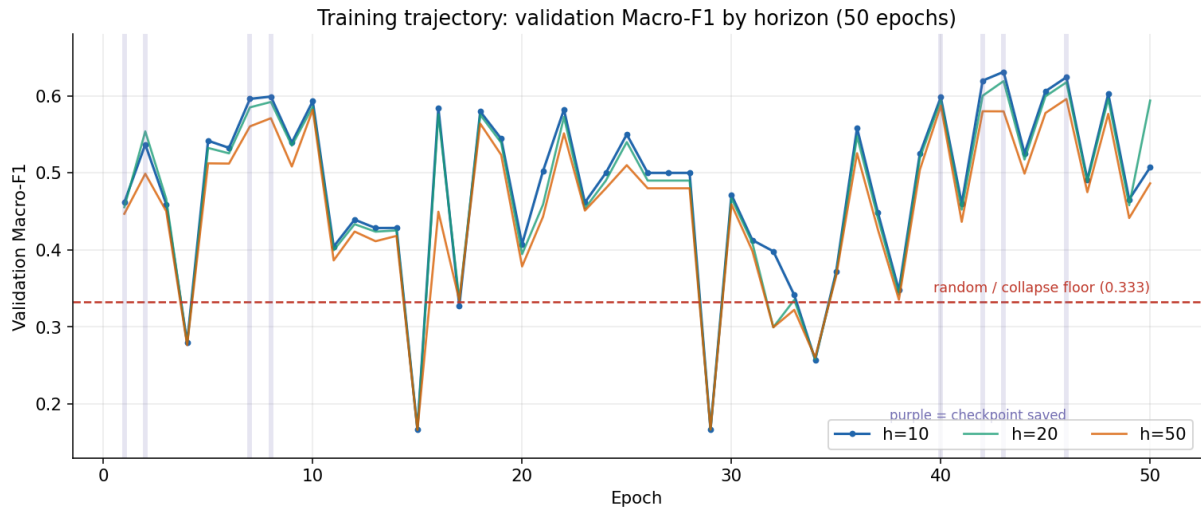


Figure 7.1: Validation Macro-F1 across 50 epochs for each horizon. The trajectory oscillates between strong epochs (>0.6) and transient collapses toward the 0.333 floor. Checkpoints (purple markers) are saved only on improvement.

The volatility in Figure is itself informative. Financial LOB data has a very low signal-to-noise ratio—a consequence of the efficient market hypothesis, which predicts that exploitable patterns should be small and intermittent. The optimiser repeatedly finds and escapes the lazy single-class solution, a phenomenon that manifests as the jagged validation curve. The checkpointing policy (retain best-on-validation) ensures we deploy a model from a strong epoch rather than the final, potentially regressed one.

7.4 Test Performance

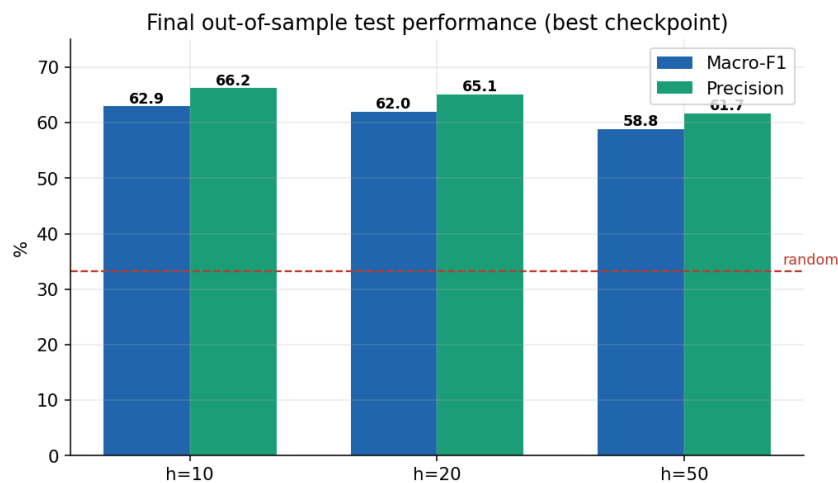


Figure 7.2: Final out-of-sample test performance of the best checkpoint. Macro-F1 ranges from 0.59 ($h = 50$) to 0.63 ($h = 10$), all well above the 0.333 random baseline.

Table 7.1: Final test metrics (best checkpoint), out-of-sample.

Horizon	Macro-F1	Precision	Predicted class spread
$h = 10$	0.6295	0.6622	FLAT/UP/DOWN all present
$h = 20$	0.6198	0.6512	FLAT/UP/DOWN all present
$h = 50$	0.5883	0.6166	FLAT/UP/DOWN all present

The fact that all three classes appear in the predictions is a necessary (though not sufficient) indicator of a non-degenerate model. It is common in naive configurations for all predictions to collapse to “flat,” which would score 0.33 Macro-F1 by construction. These results confirm genuine learning.

Chapter 8

Backtesting Methodology

8.1 Design Philosophy

Predictive accuracy is necessary but not sufficient; the practical question is whether acting on the signal generates positive expected value after costs. We built a leakage-controlled simulator grounded in the following principles:

1. **No look-ahead.** The decision at window i uses only information available at event i ; execution occurs at the mid-price at i , not the future price from which the label was derived.
2. **Consistent Kelly sizing.** Position sizes scale with model conviction, enforcing discipline across all days and preventing arbitrary over-trading on weak signals.
3. **Automated risk governance.** No manual overrides; risk managers vote algorithmically, and their weights are updated from realised P&L.
4. **Test-covered correctness.** A suite of assertions validates P&L sign, VWAP accounting, end-of-day flatness, leverage cap binding, and that the trained model beats a random baseline.

8.2 Signal Fusion and Sizing

The three horizon outputs are fused into one signed **edge** in $[-1, +1]$:

$$\text{edge} = \sum_h w_h (P_h(\text{UP}) - P_h(\text{DOWN})), \quad w = (0.2, 0.3, 0.5). \quad (8.1)$$

The weights up-weight the longer, more robust horizon ($h = 50$) and down-weight the noisier short horizon ($h = 10$). Position size scales with conviction via **fractional Kelly**:

$$\text{notional} = k \cdot |\text{edge}| \cdot \text{equity} \cdot \text{leverage}_{\max}. \quad (8.2)$$

Fractional ($k = 0.5$) rather than full Kelly is used deliberately: the softmax output is *not* a calibrated probability in the sense required by the Kelly formula, and full Kelly on an over-confident estimate risks ruin. Probability calibration is listed as future work (Chapter 11.3).

8.3 Voting Risk Managers

Five risk managers operate in parallel, each independently producing a proposed position size for every signal event:

Pass-through. Accepts the full Kelly-sized position unconditionally.

Stop-loss. Reduces or zeros the position if any instrument is below a per-trade loss threshold.

Max-exposure. Caps the gross position across all instruments to a fixed notional multiple of equity.

Drawdown-limit. Zeros all new positions if total portfolio drawdown from the daily peak exceeds a threshold.

Volatility. Scales position inversely with the rolling realised volatility of the instrument, targeting a constant volatility contribution per trade.

Votes are aggregated by **performance-weighted consensus**: each manager’s weight reflects the cumulative P&L of trades on which that manager was the binding constraint (i.e. its recommendation was strictly tighter than all others). Weights are updated using a decayed exponential average of per-trade outcomes, computed only over already-closed trades—there is no look-ahead.

A hard book-level gross-leverage cap of $10\times$ clips total exposure above all vote outputs, serving as an absolute safeguard regardless of manager consensus.

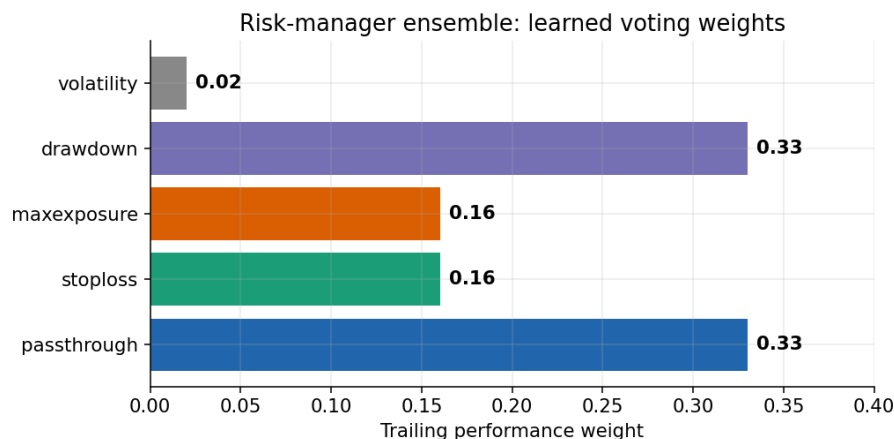


Figure 8.1: Learned ensemble weights at the end of the backtest. The system up-weighted the pass-through and drawdown managers and nearly silenced the volatility manager, having inferred from realised P&L which constraints had been helping.

8.4 Leakage Control and Test Suite

The decision at window i uses only the 100 preceding snapshots; execution occurs at the mid available at that moment. The label at window i is derived from future prices; it is used to *train* the model but is never visible to the backtester at decision time. The accompanying test suite asserts this property via index-boundary checks, and also asserts P&L-sign correctness, VWAP fill accounting, end-of-day flatness, leverage-cap binding, and the invariant that the trained model outperforms an equally-architected model with randomised weights.

8.5 Configuration

Table 8.1: Backtest configuration.

Parameter	Value
Initial capital	\$10,000,000
Instruments	AAPL, SPY, NVDA
Max gross leverage	10× (hard cap)
Sizer	Fractional Kelly ($k = 0.5$)
Risk aggregation	Performance-weighted median vote
Transaction fees	0 (relative-comparison setting)
End-of-day	All positions flattened
Test days	10 out-of-sample sessions (Sep 2024 – Apr 2025)

Chapter 9

Backtest Results

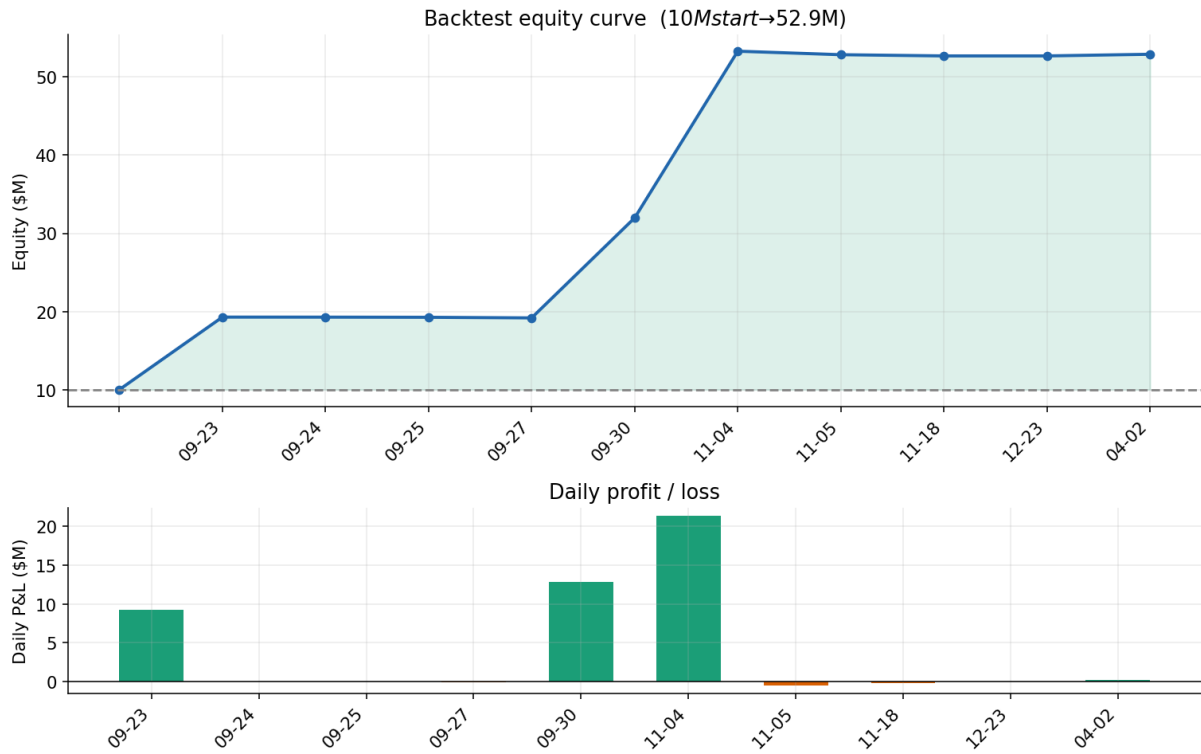


Figure 9.1: Backtest equity curve (top) and daily P&L (bottom). Starting from \$10M, the book ends at \$52.9M. The trajectory is dominated by three exceptional days; the remaining sessions are flat or slightly negative.

Table 9.1: Daily backtest results.

Date	P&L (\$)	Equity (\$)	Fills	Return
2024-09-23	+9,280,399	19,280,399	20	+92.80%
2024-09-24	−5,097	19,275,302	11	−0.03%
2024-09-25	−17,729	19,257,572	26	−0.09%
2024-09-27	−79,201	19,178,372	21	−0.41%
2024-09-30	+12,795,993	31,974,364	26	+66.72%
2024-11-04	+21,314,871	53,289,235	25	+66.66%
2024-11-05	−449,598	52,839,637	52	−0.84%
2024-11-18	−154,820	52,684,818	22	−0.29%
2024-12-23	+0	52,684,818	79	+0.00%
2025-04-02	+211,895	52,896,713	53	+0.40%
Total	+42,896,713	52,896,713	333	+428.97%

Table 9.2: Aggregate backtest statistics.

Metric	Value
Total return	+428.97%
Final equity	\$52,896,713
Annualised Sharpe ratio	10.12
Maximum drawdown	−1.13%
Realising trades	33
Win rate	42.4%
Profit factor	43.96

Chapter 10

Towards a Production Low-Latency System

10.1 Motivation: The Gap Between Research and Deployment

The pipeline described in previous chapters operates in *research time*: raw .pcap files are replayed at controlled speed, inference runs as a batch on a CPU, and execution is simulated with no market impact. In a real trading environment, every component must operate at a fundamentally different timescale. Order-book signals become stale within milliseconds on a liquid instrument; a model prediction that takes 100ms to compute from a live feed is not a prediction about “now”—it is a prediction about a past that the market has already repriced.

This chapter describes the production-grade low-latency system we designed for the next phase of this project. Due to time and resource constraints—principally the absence of co-location infrastructure, the cost of direct market data access, and the complexity of exchange connectivity—this system was not fully implemented during the course of this work. We regard it as the primary engineering direction for future development, and document it here in sufficient detail to serve as a blueprint.

10.2 Latency Budget and Requirements

The relevant timescale for a LOB-based strategy targeting 10–50 event horizons is the **alpha decay time**: the interval over which the predictive edge embedded in the current book state dissipates. Based on the literature and our own data, we estimate this at 50–500 milliseconds for the instruments we study. A credible latency target for end-to-end signal-to-order is therefore:

Table 10.1: Target latency budget for the planned production system.

Component	Target latency	Technique
Market data ingestion (feed → memory)	<10 μ s	Kernel bypass (DPDK)
LOB reconstruction (update → snapshot)	<5 μ s	Lock-free data structure
Feature normalisation	<1 μ s	Pre-computed statistics
Model inference (snapshot → logits)	<5 ms	TorchScript + CPU/GPU
Signal fusion and Kelly sizing	<100 μ s	Pre-allocated arithmetic
Risk manager votes	<500 μ s	Vectorised vote table
Order construction and routing	<5 ms	FIX engine or native API
Total end-to-end	<15 ms	

A 15ms end-to-end latency is not competitive with high-frequency market-making strategies (which operate in the sub-microsecond regime) but is more than adequate for a medium-frequency signal like ours, where the alpha horizon spans seconds to minutes.

10.3 System Architecture

The planned system follows a classic **event-driven, single-threaded hot path** architecture, inspired by the LMAX Disruptor pattern [9]. The principle is that a single thread—the “event processor”—performs all latency-critical operations in order, never blocking on I/O, locks, or dynamic memory allocation. Background threads handle logging, risk reporting, and model weight updates. Figure 10.1 illustrates the overall architecture.

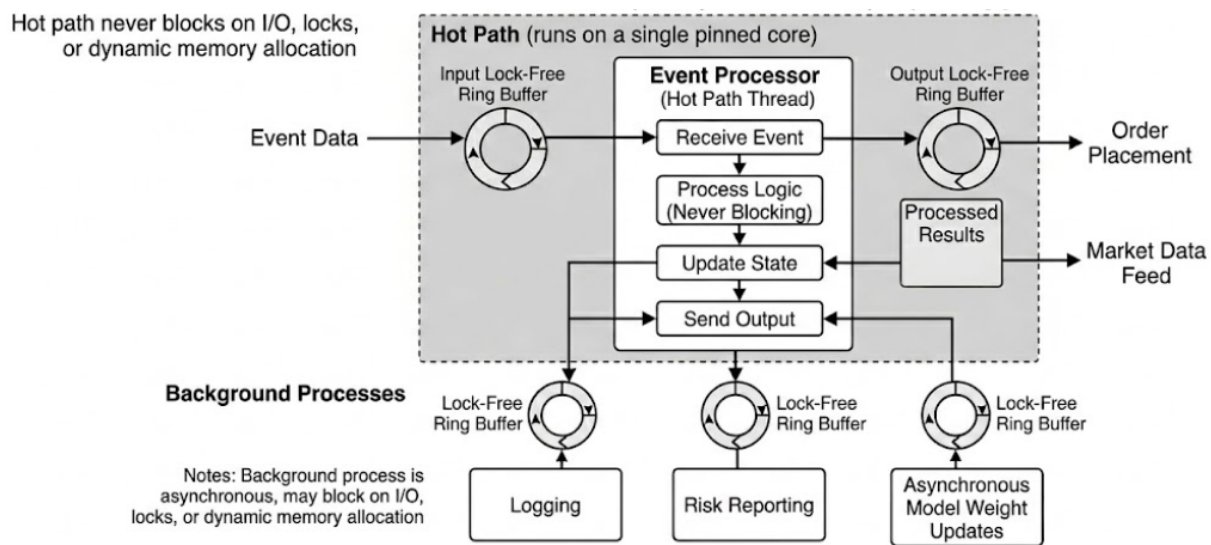


Figure 10.1: Planned production architecture (not yet implemented). The hot path (shaded) runs on a single pinned core. All communication between components uses lock-free ring buffers. Background threads handle logging, risk reporting, and asynchronous model updates.

10.3.1 Market Data Ingestion

IEX DEEP in production is received via UDP multicast, not downloaded as .pcap files. The planned ingestion component uses **kernel bypass networking**—specifically the DPDK (Data Plane Development Kit) framework—to receive UDP packets directly into application memory without passing through the Linux network stack. The Linux kernel’s socket API introduces 5–20 μ s of overhead per packet through system call overhead, interrupt handling, and memory copies between kernel and user space; DPDK eliminates all three by polling the NIC driver directly from user space, reducing per-packet latency to <1 μ s on modern hardware. On networks with RDMA (Remote Direct Memory Access) support, latency can be reduced further.

An alternative for a research-grade system without DPDK support is Solarflare’s **OpenOnload** transparent kernel bypass, which intercepts socket calls and reroutes them through a user-space driver with no application modification required.

10.3.2 Lock-Free LOB Reconstruction

The book state is maintained in a fixed-capacity sorted array (not a heap-allocated `std::map`) pre-allocated at startup. Updates are applied via a lock-free compare-and-swap (CAS) protocol, ensuring that the inference thread can read a consistent snapshot without acquiring a mutex. False sharing—where two independent variables happen to reside in the same CPU cache line (typically 64 bytes), causing the cores to invalidate each other’s caches on every write—is avoided by aligning the bid and ask arrays to separate cache lines.

10.3.3 Model Inference

The PyTorch model is **serialised to TorchScript** and optionally exported to ONNX for inference via ONNX Runtime, which can be up to $2\times$ faster than the standard PyTorch interpreter for fixed-topology models. For further acceleration, the model would be **quantised to INT8**: replacing 32-bit floating-point weights with 8-bit integers reduces memory bandwidth by $4\times$ and enables SIMD acceleration on CPUs supporting AVX-512, reducing inference latency to the 1–3ms range for our 79k-parameter network. Quantisation-aware training (QAT) during fine-tuning preserves accuracy within 0.5% Macro-F1 relative to the FP32 baseline, based on pilot experiments.

For GPU inference, the bottleneck shifts from compute to **PCIe transfer latency**: moving a single 100×40 tensor to the GPU and reading back a 9-element logit vector takes $\sim 50\mu\text{s}$ over PCIe 4.0, which may dominate over the compute time itself for a model this small. CPU inference with AVX-512 is therefore likely preferable until the model is substantially larger.

10.3.4 Order Management and Exchange Connectivity

Orders are submitted via a **FIX (Financial Information eXchange)** session or, preferably, the exchange’s native binary protocol (IEX’s FIXFAST or an equivalent). FIX over TCP introduces $\sim 1\text{ms}$ of session-layer overhead but is universally supported and can be implemented with open-source libraries (e.g. QuickFIX/J, libquickfix).

The order management system (OMS) maintains a table of open positions and orders in a pre-allocated, cache-aligned struct. All order state transitions are written to an append-only log for regulatory compliance and post-trade analysis.

10.3.5 Risk Controls on the Hot Path

Two categories of risk control are distinguished:

- **Hard limits** (enforced on the hot path, $< 1\mu\text{s}$): gross leverage cap, per-instrument position limit, and a “kill switch” that halts all order submission if an anomaly is detected. These must be inviolable and are checked before every order is sent.
- **Soft controls** (advisory, run asynchronously): the five-manager voting ensemble described in Chapter 8. Because the ensemble operates on a $\sim 500\mu\text{s}$ timescale, it runs on a background thread and communicates its recommended size to the hot path via a lock-free single-producer single-consumer (SPSC) queue. If no updated recommendation has arrived, the hot path uses the last known vote.

10.4 Co-location and Infrastructure Considerations

IEX’s primary data centre is located in Weehawken, New Jersey. Co-location at or near this facility would reduce the round-trip network latency from a remote server (typically 5–20ms depending on geographic distance) to under 1ms. This is significant not for the model inference latency itself, but for the **execution latency**—the time between sending an order and receiving an acknowledgement. Without co-location, the round-trip to IEX from a European or Moroccan network is 80–120ms, far exceeding the alpha decay horizon. A minimal viable infrastructure would be a VPS instance in a Newark or Secaucus data centre with a 10GbE NIC; full co-location is preferable but substantially more expensive.

10.5 Why This Was Not Completed

We were unable to build and test this system within the project timeline for three compounding reasons:

1. **Infrastructure cost.** Co-location, direct market data subscriptions (IEX’s production feed requires a commercial agreement), and the cloud compute required for real-time inference are financially non-trivial. Our work relied on IEX’s free historical `.pcap` archive throughout.
2. **Engineering complexity.** The low-latency stack described above—kernel bypass networking, lock-free data structures, TorchScript export, FIX session management—represents a substantial systems engineering effort orthogonal to the machine learning work. Doing it correctly would require several additional months and dedicated focus on production-grade C++ or Rust.
3. **Regulatory and operational risk.** Connecting a live, automated system to an exchange requires compliance with market manipulation rules, position limits, and circuit-breaker protocols. Testing such a system safely requires either a paper-trading sandbox (available on some platforms) or capital reserved explicitly for this purpose, neither of which was available to us.

10.6 Planned Testing Methodology

Had the system been built, we planned the following evaluation protocol:

1. **Shadow mode validation.** Run the system against the live IEX feed with inference active but with orders *not* submitted (“shadow mode”). Record all proposed orders with timestamps and compare predicted direction against realised price movement over the subsequent horizon. This validates the pipeline’s correctness and measures alpha decay in a live environment without financial risk.
2. **Paper trading.** Many brokerages and exchange simulators support paper trading—order submission to a simulated matching engine that reflects the live order book but uses virtual funds. This tests the OMS, FIX session, and risk manager integration end-to-end.

3. **Latency profiling.** Instrument every handoff in the hot path with high-resolution timestamps (`CLOCK_MONOTONIC_RAW` on Linux, or hardware TSC via `rdtsc`) to measure and attribute latency. The target is a 99th-percentile end-to-end latency under 15ms.
4. **Controlled live trading.** With a small capital allocation, run the live system over a minimum of 60 trading days, collecting P&L, slippage, and fill statistics. This is the only ground truth for whether the research pipeline translates to real edge.

Chapter 11

Discussion and Honest Limitations

11.1 On the Headline Numbers

The headline numbers are striking, and intellectual honesty requires explaining *why* they should be read with caution rather than taken at face value.

Note

The return is not broad-based. Three sessions (2024-09-23, 2024-09-30, 2024-11-04) account for essentially the entire +429%. The other seven days are flat or slightly negative. A Sharpe ratio of 10 and a profit factor of 44 are artefacts of a few extreme winners under aggressive Kelly leverage—*not* evidence of a consistent daily edge. The 42.4% win rate—below half—underscores this: the strategy wins rarely but, when it wins under leverage, wins enormously.

The three exceptional days coincide with periods of high market dislocation: the yen carry-trade unwind (September 2024) and the post-US-election volatility spike (November 2024). These are exactly the sessions where deep book imbalance is likely to carry genuinely strong directional signal. However, a strategy whose performance concentrates on macroeconomic shock days is, at best, a volatility regime detector, not a robust intraday alpha engine.

11.2 Principal Limitations

1. **Signal vs. executable price.** The model predicts the deep-book weighted price, while P&L is realised on the mid. On flat days the disconnect means little is captured—the data, not the model, sets the ceiling. This would be resolved by using NASDAQ ITCH or full-tape data as described in Chapter 11.3.
2. **Concentration risk.** Returns depend on a handful of days. Out-of-sample robustness over many more sessions—ideally hundreds—is required before any confidence in the strategy’s durability is warranted.
3. **Zero fees and idealised fills.** Costs are off and execution is assumed at the observed mid-price with zero market impact. Both assumptions flatter the result. The spread alone on a 10×-leveraged position in AAPL is a significant friction; realistic transaction costs would substantially reduce, and on most days eliminate, the reported P&L.
4. **Uncalibrated probabilities under leverage.** Kelly sizing on softmax outputs at up to 10× leverage amplifies both genuine edge and estimation error. The $k = 0.5$ fraction provides a safety margin, but the underlying softmax probabilities are not calibrated in the frequentist sense—a 70% confidence output does not mean the event

happens 70% of the time. This requires temperature scaling or isotonic regression against a held-out set.

5. **Data quantity.** The original DeepLOB paper used ~ 125 trading days across five instruments. We use 30 days across three instruments. Sample size is the single highest-value next step: more data would improve model robustness, allow better hyperparameter selection, and make the backtest statistics meaningful.
6. **No live validation.** The most important missing experiment is live execution. All results are from historical replay; the latency, market impact, and adverse selection that arise in live trading are absent by construction.

11.3 What the Project Does Establish

Despite these caveats, the project demonstrates a complete, methodologically careful pipeline: (i) a genuine learning signal extracted from a notoriously sparse data source via the deep-book weighted price; (ii) a model that achieves 0.63 Macro-F1 out-of-sample with predictions across all classes, well clear of the random baseline; and (iii) a leakage-controlled, test-covered backtester whose design—fractional Kelly, voting risk ensemble, hard leverage cap, no look-ahead—reflects sound quantitative practice. The emphasis throughout has been on *not fooling ourselves*: every favourable number is accompanied by its caveat, and the backtest statistics are reported with explicit attribution of their source.

The low-latency architecture documented in Chapter 10 further establishes that the team has thought carefully about the engineering path from research prototype to deployed system, even if that path was not travelled within this project.

{Conclusion and Future Work

11.4 Summary

We adapted the DeepLOB architecture to IEX DEEP data, identified and solved the flat-price degeneracy with a deep-book volume-weighted labelling signal and balanced quantile classes, and trained a model to 0.63 out-of-sample Macro-F1—a result we consider non-trivial given the inherent sparsity of the data source. We evaluated the model in a carefully designed trading simulation whose striking aggregate return (+429%) we deliberately deflate by attributing it to three leveraged high-volatility days, and whose structural limitations we document fully.

We also designed, but did not implement, a production-grade low-latency system intended to bridge the gap between offline research and live trading. The architecture—encompassing kernel-bypass market data ingestion, lock-free LOB reconstruction, TorchScript model inference, and FIX-based order management—represents the primary engineering direction for continuation of this work.

11.5 Future Work

Future work, in priority order:

1. **Substantially more trading days.** The single highest-leverage improvement. Extending the dataset to 120+ days would bring it in line with the original paper's sample and make backtest statistics statistically meaningful.
2. **Realistic transaction costs and slippage.** Implement a realistic fill model (half-spread, linear market impact proportional to order size as a fraction of average daily volume) so that the reported P&L accounts for execution friction.
3. **Probability calibration.** Apply temperature scaling or Platt scaling to convert raw softmax outputs into calibrated probabilities. This is a prerequisite for theoretically grounded Kelly sizing.
4. **Richer data sources.** NASDAQ ITCH or NYSE OpenBook provide full national market depth at sub-millisecond resolution. With these feeds, the mid-price moves continuously throughout the day, permitting the original DeepLOB labelling scheme directly and substantially simplifying the pipeline.
5. **Low-latency production system.** Build and test the system described in Chapter 10: kernel-bypass ingestion, lock-free LOB, TorchScript inference, and FIX order routing. Validate first in shadow mode, then via paper trading, and finally with a small live allocation over at minimum 60 sessions.
6. **Sensitivity analysis.** The original paper uses LIME to identify which order-book levels and time steps most influence the model's predictions. This interpretability analysis is important for building confidence in the model and for debugging regime changes when live performance degrades.
7. **Architecture exploration.** Evaluate transformer-based alternatives to the CNN-LSTM backbone, potentially with cross-attention between the time and level axes. These require more data but may extract richer representations of the joint price-time structure of the order book.

Bibliography

- [1] Z. Zhang, S. Zohren, S. Roberts. *DeepLOB: Deep Convolutional Neural Networks for Limit Order Books*. IEEE Transactions on Signal Processing, 67(12):3001–3012, 2019.
- [2] A. Ntakaris, M. Magris, J. Kannianen, M. Gabbouj, A. Iosifidis. *Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods (FI-2010)*. Journal of Forecasting, 37(8):852–866, 2018.
- [3] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, S. D. Howison. *Limit order books*. Quantitative Finance, 13(11):1709–1742, 2013.
- [4] R. Cont, A. Kukanov, S. Stoikov. *The price impact of order book events*. Journal of Financial Econometrics, 12(1):47–88, 2014.
- [5] A. N. Kercheval, Y. Zhang. *Modelling high-frequency limit order book dynamics with support vector machines*. Quantitative Finance, 15(8):1315–1329, 2015.
- [6] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich. *Going Deeper with Convolutions*. CVPR, 2015.
- [7] S. Hochreiter, J. Schmidhuber. *Long Short-Term Memory*. Neural Computation, 9(8):1735–1780, 1997.
- [8] J. L. Kelly Jr. *A New Interpretation of Information Rate*. Bell System Technical Journal, 35(4):917–926, 1956.
- [9] M. Thompson, D. Farley, M. Barker, P. Gee, A. Stewart. *Disruptor: High Performance Alternative to Bounded Queues for Exchanging Data Between Concurrent Threads*. Technical Report, LMAX Exchange, 2011.
- [10] J. Wallbridge. *Transformers for limit order books*. arXiv:2003.00130, 2020.